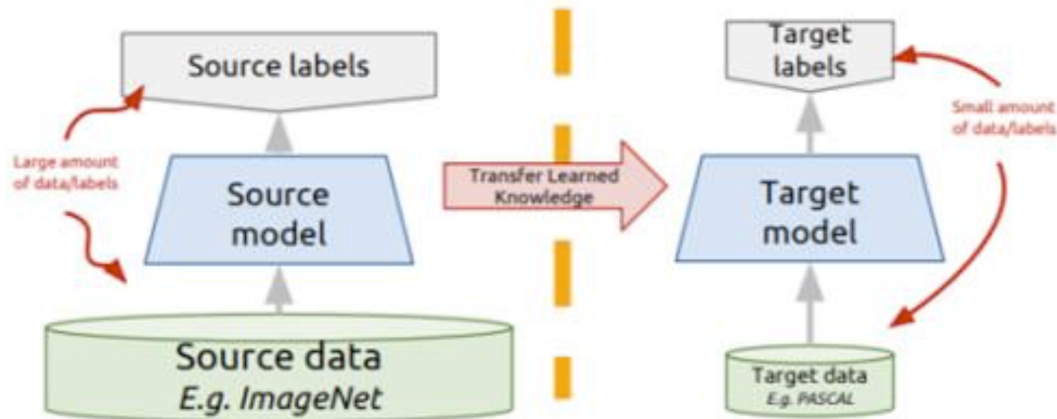


TRANSFERENCIA DE APRENDIZAJE (TRANSFER LEARNING)

Transferencia del conocimiento

Es la mejora del aprendizaje en una nueva tarea mediante la transferencia de conocimientos de una tarea relacionada que ya se ha aprendido.

El **transfer learning** es popular en el aprendizaje profundo dados los **enormes recursos necesarios** para entrenar modelos de aprendizaje profundo o los **grandes y desafiantes conjuntos de datos** en los que se entrenan estos modelos.



Transferencia del conocimiento con ImageNet

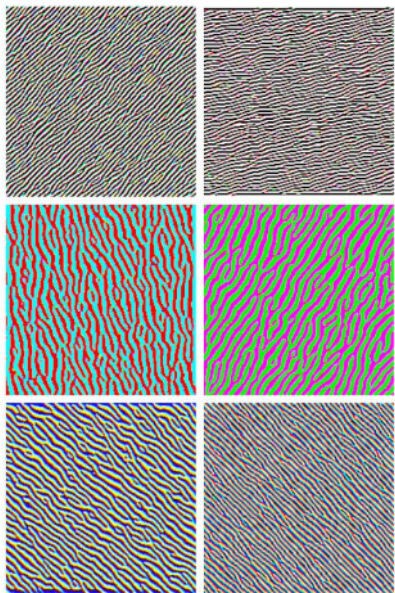
Las organizaciones de investigación que desarrollan modelos para esta competencia y lo hacen bien, a menudo, **publican su modelo final** bajo una licencia permisiva para su reutilización. Estos modelos pueden tardar **días o semanas** en entrenarse con hardware moderno.

Reutilizar estos modelos es un enfoque efectivo porque:

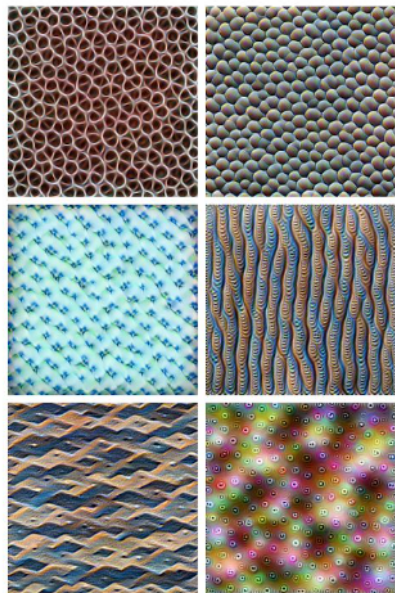
1. Fueron entrenados con un dataset ***“lo suficientemente grande”*** en el orden de los millones de ejemplos de entrenamiento.
2. Fueron entrenados para resolver una **tarea representativa** del espacio de problemas de la disciplina

Estructuras Jerárquicas de las Features

Las características de las redes neuronales profundas entrenadas en ImageNet pasan de lo general a lo específico de la primera a la última capa: las capas inferiores aprenden a modelar características de bajo nivel como bordes, mientras que las capas superiores modelan conceptos de nivel superior como patrones, partes enteras u objetos.



Edges (layer conv2d0)



Textures (layer mixed3a)



Patterns (layer mixed4a)



Parts (layers mixed4b & mixed4c)



Objects (layers mixed4d & mixed4e)

Generalización de las Features Aprendidas

Las características de las redes neuronales profundas entrenadas en ImageNet pasan de lo general a lo específico de la primera a la última capa: las capas inferiores aprenden a modelar características de bajo nivel como bordes, mientras que las capas superiores modelan conceptos de nivel superior como como patrones, partes enteras u objetos.

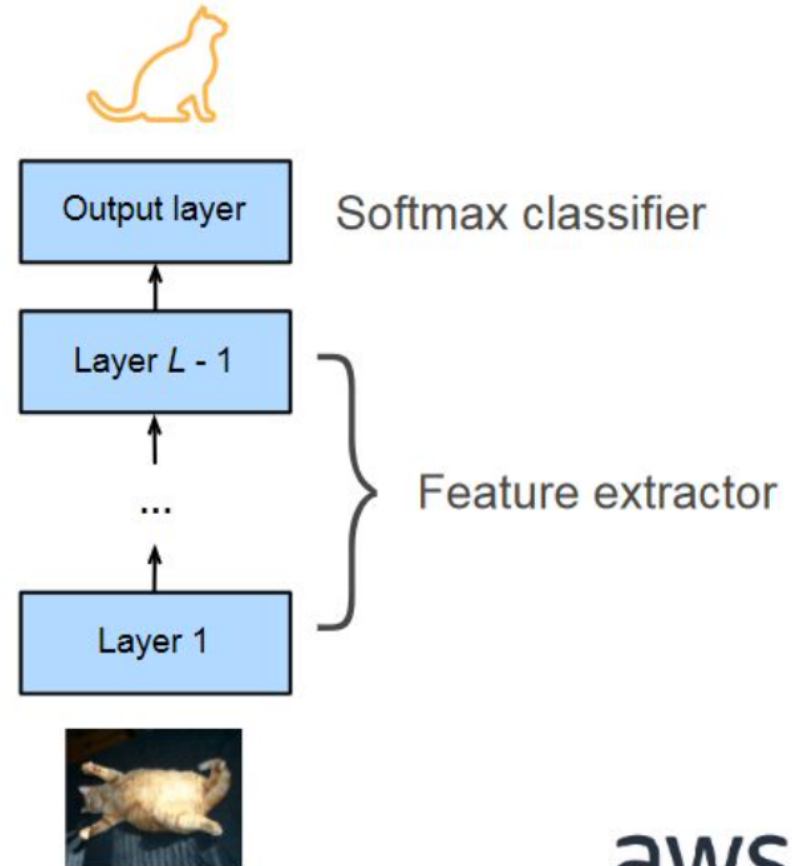


Enfoque de Fine Tuning

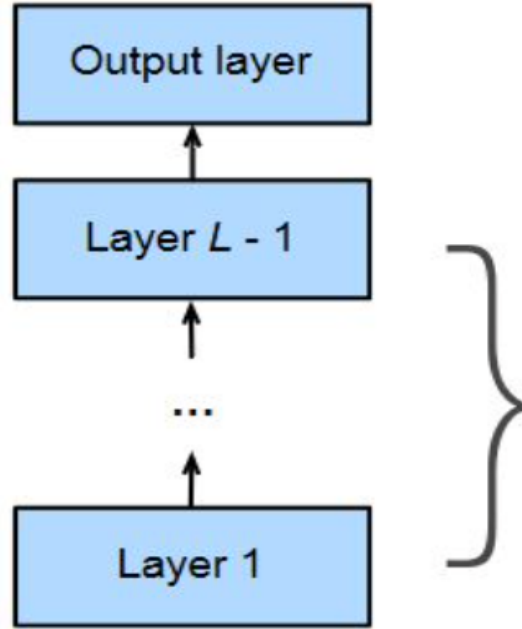
Estructura de una Red Neuronal

Una red neuronal se puede dividir aproximadamente en dos partes:

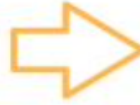
- Un **extractor de características** que mapea píxeles sin procesar en características linealmente separables
- Un **clasificador lineal** para tomar decisiones



Fine Tuning



No debería usar directamente los parámetros del clasificador porque las etiquetas cambiaron.



Quizás también sea un buen extractor para mi dataset

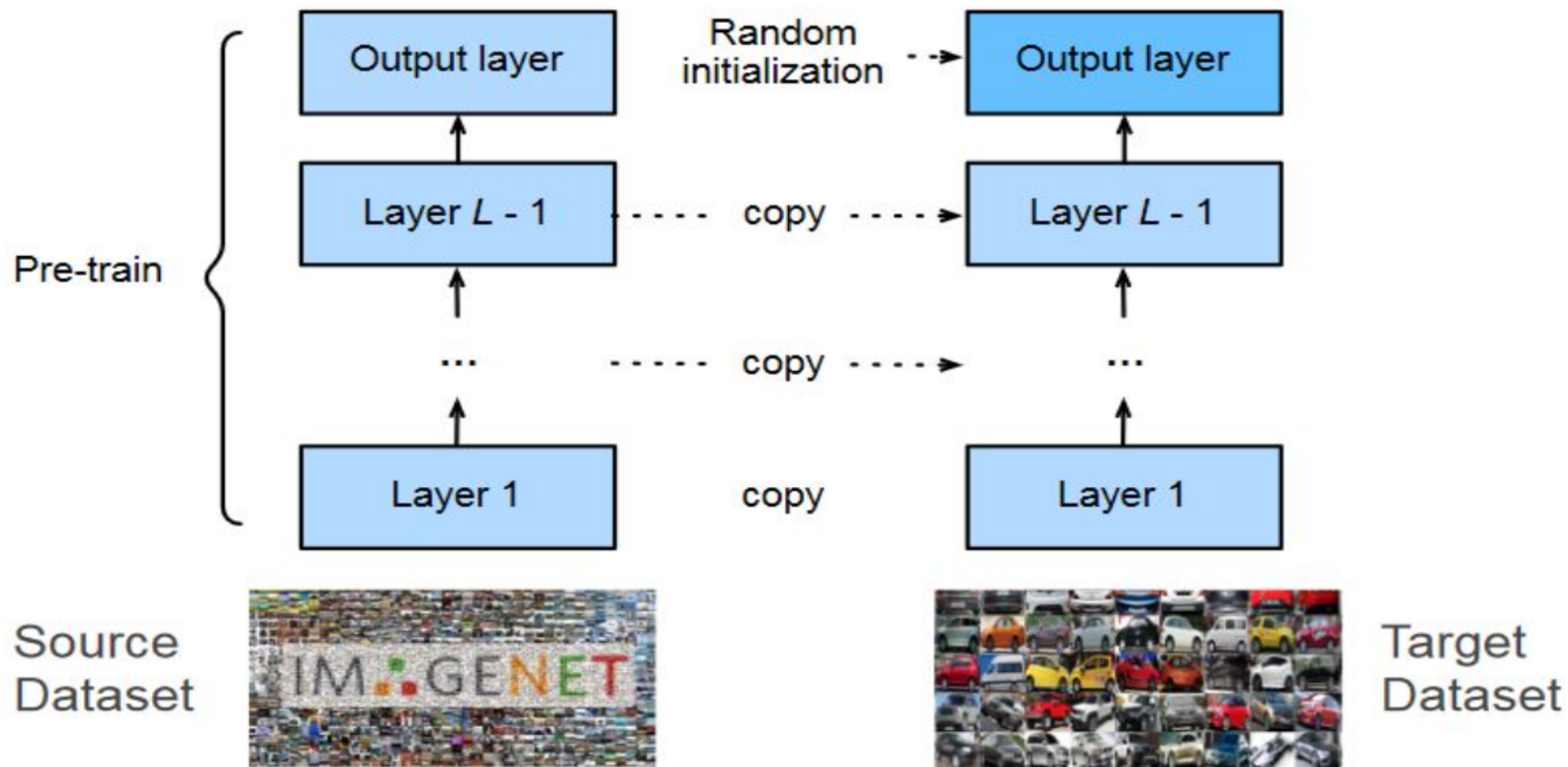
Source Dataset



Target Dataset



Fine Tuning



Consideraciones durante el entrenamiento

1. Entrene en el dataset como un proceso de entrenamiento normal, pero con una fuerte regularización
 - a. Tasa de aprendizaje pequeña
 - b. Menos épocas
 - c. Hiperpámetros de Dropout más agresivos
2. Congele las capas iniciales.

Implementación

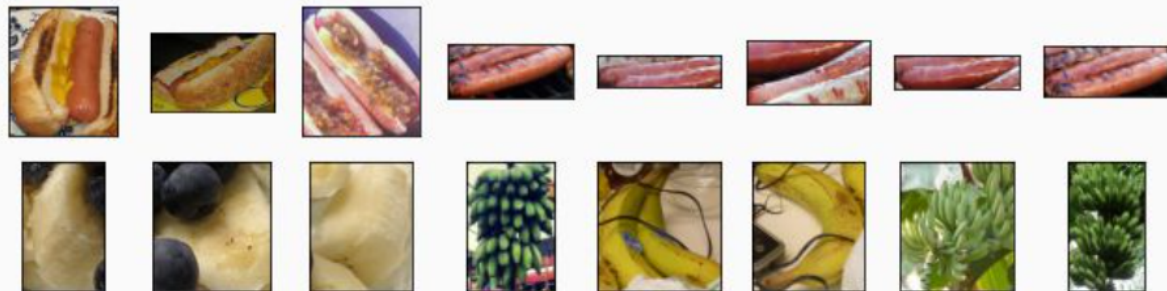
Obtener Datos Target

```
#@save
d2l.DATA_HUB['hotdog'] = (d2l.DATA_URL + 'hotdog.zip',
                           'fba480ffa8aa7e0febbb511d181409f899b9baa5')

data_dir = d2l.download_extract('hotdog')
```

```
train_imgs = torchvision.datasets.ImageFolder(os.path.join(data_dir, 'train'))
test_imgs = torchvision.datasets.ImageFolder(os.path.join(data_dir, 'test'))
```

```
hotdogs = [train_imgs[i][0] for i in range(8)]
not_hotdogs = [train_imgs[-i - 1][0] for i in range(8)]
d2l.show_images(hotdogs + not_hotdogs, 2, 8, scale=1.4);
```



Aumentar los Datos

```
train_augs = torchvision.transforms.Compose([  
    torchvision.transforms.RandomResizedCrop(224),  
    torchvision.transforms.RandomHorizontalFlip(),  
    torchvision.transforms.ToTensor(), normalize])
```

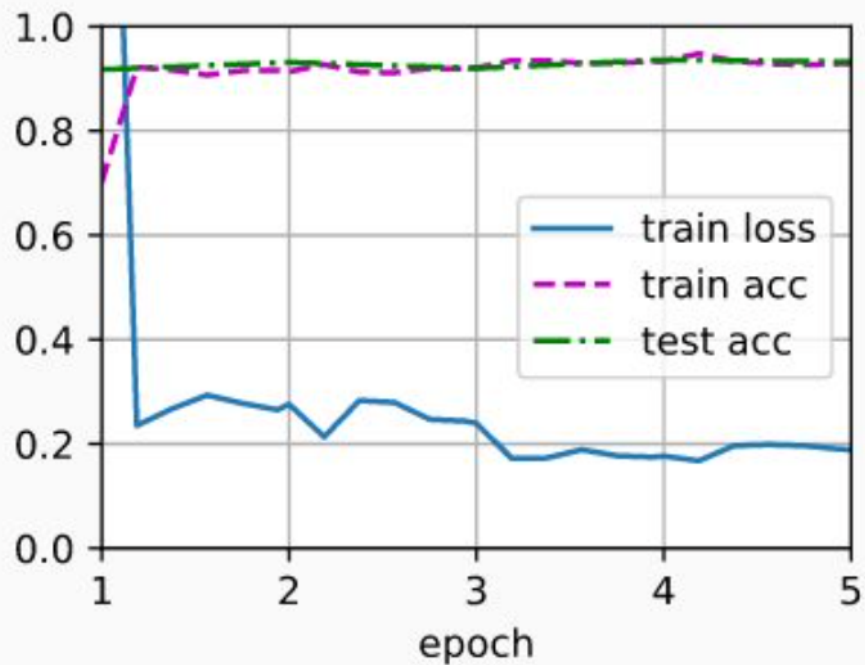
```
test_augs = torchvision.transforms.Compose([  
    torchvision.transforms.Resize(256),  
    torchvision.transforms.CenterCrop(224),  
    torchvision.transforms.ToTensor(), normalize])
```

Fine Tuning

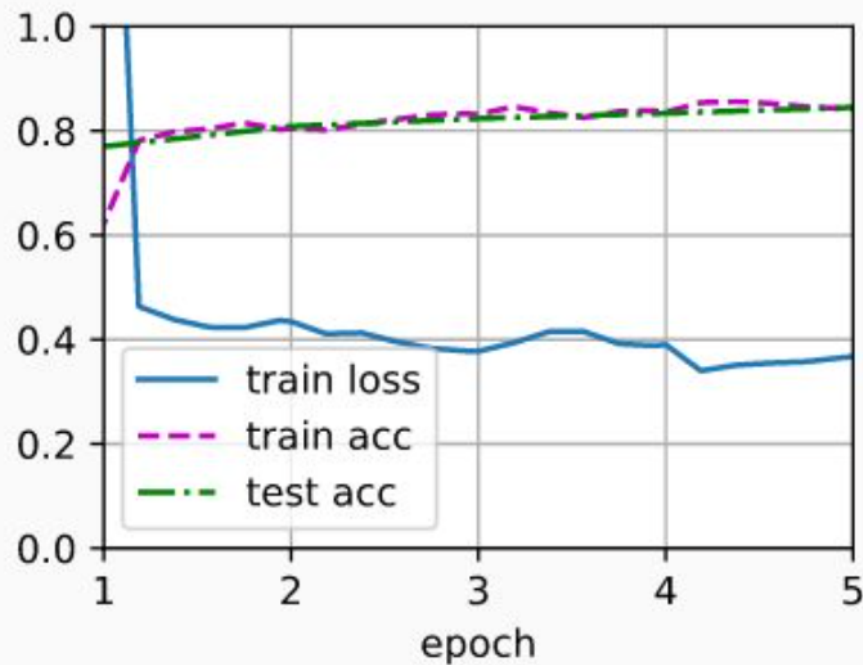
```
finetune_net = torchvision.models.resnet18(pretrained=True)
finetune_net.fc = nn.Linear(finetune_net.fc.in_features, 2)
nn.init.xavier_uniform_(finetune_net.fc.weight)
train_iter = torch.utils.data.DataLoader(
    torchvision.datasets.ImageFolder(os.path.join(data_dir, 'train'),
                                     transform=train_augs),
    batch_size=batch_size, shuffle=True)
test_iter = torch.utils.data.DataLoader(
    torchvision.datasets.ImageFolder(os.path.join(data_dir, 'test'),
                                     transform=test_augs),
    batch_size=batch_size)
devices = d2l.try_all_gpus()
loss = nn.CrossEntropyLoss(reduction="none")
if param_group:
    params_1x = [
        param for name, param in net.named_parameters()
        if name not in ["fc.weight", "fc.bias"]]
    trainer = torch.optim.SGD([
        {'params': params_1x}, {
            'params': net.fc.parameters(),
            'lr': learning_rate * 10}], lr=learning_rate,
                              weight_decay=0.001)
else:
    trainer = torch.optim.SGD(net.parameters(), lr=learning_rate,
                              weight_decay=0.001)
```

Resultados

Con Fine Tuning



Sin Fine Tuning



PREGUNTAS???

GRACIAS POR SU ATENCIÓN!!!